

7.6.2 Weiterführende Literatur



[Riehle97] beschreibt den Entwurf objektorientierter Oberflächen.

[Coldewey97] und [Coldewey98] stellen Entwurfsmuster für grafische Oberflächen vor.

[Cai99] beschreiben den Komponentenentwurf eines Fat-Client mit einer hierarchischen MVC-Struktur.

[Buschmann96] führt das MVC Muster aus.

Kerstin Dittert (kerstin.dittert@oocon.de) ist freie Beraterin mit den Schwerpunkten Software-Architektur, OO-Methodik, J2EE und Projektmanagement. Sie unterstützt mittlere und große Unternehmen in allen Gebieten des objektorientierten Softwareentwicklungsprozesses sowie in Technologiefragen. Hierbei ist sie unter anderem als Software-Architektin, Analytikerin, Designerin, Projektleiterin sowie als Trainerin und Coach tätig (www.oocon.de).

■ 7.7 Ergonomie grafischer Oberflächen

von Kerstin Dittert

Grafische Benutzeroberflächen (GUI) bieten im Vergleich zu alphanumerischen Benutzerschnittstellen viele Gestaltungsmöglichkeiten. Dies betrifft sowohl die Arbeitsabläufe als auch die Art der Darstellung.

Wir stellen Ihnen in diesem Abschnitt die wichtigsten Aspekte der Gestaltung von Benutzeroberflächen vor:

- Arbeitsmetaphern veranschaulichen die Struktur von Arbeitsabläufen.
- Interaktionsstile bestimmen die Art der grafischen Darstellung.
- Die Berücksichtigung ergonomischer Aspekte steigert die Effektivität, Effizienz und Akzeptanz Ihres Systems.

7.7.1 Arbeitsmetaphern

Metapher =
Analogie

Metaphern bieten uns Bilder aus der wirklichen Welt an, um die Struktur von Arbeitsabläufen und Anwender-Interaktionen zu veranschaulichen. Dabei deckt eine Metapher nicht immer alle Anwendungsfälle eines Systems ab. Das Gesamtsystem stellt eine ausgewogene Mischung einzelner Metaphern dar.

Fabrik-Metapher

In der Fabrik erfolgt ein Arbeitsschritt nach dem anderen. Das Ausgangsprodukt eines Bearbeitungsschrittes wird zum Eingangsprodukt des nächsten Arbeitsschrittes. Die Produktionsbänder einer Fabrik können sich verzweigen und wieder vereinigen. Jedes Produkt folgt jedoch nur einem Weg vom Ausgangsprodukt zum Endprodukt. Auf diesem Weg ist die Reihenfolge der Arbeitsschritte fest vorgegeben.

Fabrik

Häufig gibt es zwischen den einzelnen Schritten eine Qualitätskontrolle, die bisherige Ergebnisse überprüft und fehlerhafte Zwischenergebnisse ablehnt. Sind alle Arbeitsschritte durchlaufen, so liegt ein fertiges Produkt vor.

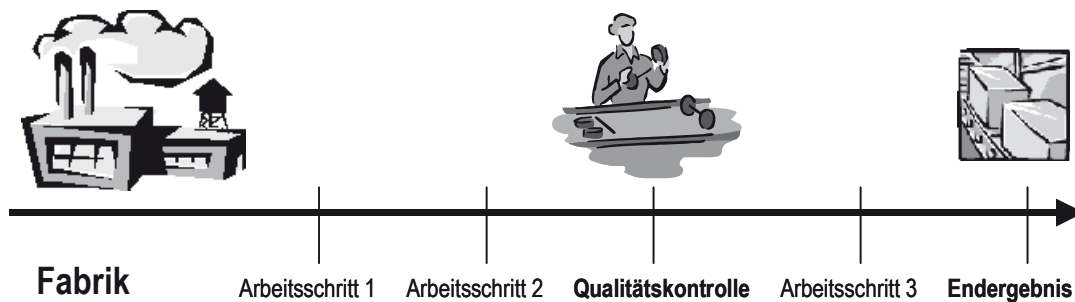


BILD 7.27 Fabrik-Metapher

Der Arbeitsablauf folgt starren Regeln. Abkürzungen auf dem Weg zum gewünschten Endprodukt sind in der Regel nicht möglich. Die starre Vorgabe begünstigt eine leichte Erlernbarkeit der Arbeitsschritte und führt zu einem hohen Durchsatz. Die Arbeit des einzelnen Anwenders ist auf effiziente Massenverarbeitung ausgerichtet und in der Regel wenig kreativ. Die Fabrik-Metapher ist passend für Systeme, die

Hoher
Durchsatz

Leicht erlernbar



- selten verwendet werden und ihre Nutzer daher an den Arbeitsablauf „erinnern“ müssen;
- von Anwendern benutzt werden, die nur wenig Erfahrung im Umgang mit EDV-Systemen haben;
- Massendaten verarbeiten oder
- einen stark sequenziellen Arbeitslauf haben (etwa Internet-Shop).

Werkzeug-Material-Metapher

Das Leitbild dieser Metapher ist ein Arbeitsplatz für qualifizierte Tätigkeiten, bei denen der Mensch im Mittelpunkt steht. Das Umfeld setzt die entsprechende Qualifikation zur Bewältigung der Aufgaben voraus. Die Arbeitsumgebung ist auf individuelle Arbeitsschwerpunkte und Erfahrung des Einzelnen hin anpassbar. Die fachlichen Konzepte sind klar erkennbar und unterstützen kreative Arbeit und komplexe Tätigkeiten.

Werkzeug-
Material

Der Arbeitsablauf ist wenig strukturiert. Viele Kombinationsmöglichkeiten von Werkzeug und Material sowie eine beliebige Reihenfolge der Arbeitsschritte erlauben unterschiedliche Wege zum Ziel. In der Domäne werden Werkzeuge, Material und der Verwendungszusammenhang unterschieden. Diese entsprechen den Arbeitsmitteln, den Arbeitsgegenständen und den Tätigkeiten. Eine Tätigkeit besteht in der Anwendung eines Werkzeugs auf das zu bearbeitende Material. Die Prüfung der Arbeitsergebnisse kann implizit nach Anwendung eines Arbeitsschrittes erfolgen, oder explizit auf Anforderung des Bearbeiters.

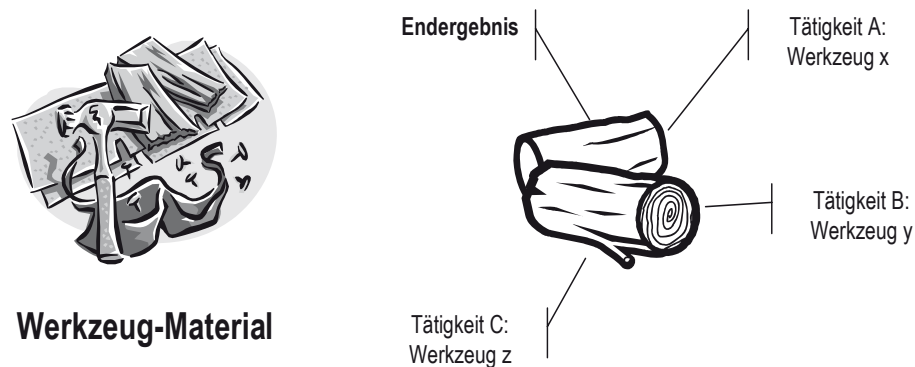


BILD 7.28 Werkzeug-Material-Metapher

Die Werkzeug-Material-Metapher eignet sich für Anwendungen, die



- unterschiedliche Arbeitsabläufe für gleichartige Aufgaben abbilden;
- kreative Tätigkeiten durch eine Vielzahl von Wahlmöglichkeiten unterstützen (beispielsweise Bildbearbeitung);
- von Experten benutzt werden, die das System entsprechend ihrer individuellen Fähigkeiten optimal nutzen möchten.

Formular-Metapher

Ein Formular enthält eine Vielzahl von Daten, die in verdichteter Form dargestellt werden. Zum Ausfüllen eines Formulars ist häufig Expertenwissen über verwendete Abkürzungen, Schlüsselbegriffe und die Reihenfolge der Bearbeitung erforderlich. Die Daten werden überwiegend als Text dargestellt.

Beispiele für derartige Formulare sind Versicherungsanträge, Steuererklärungen, Schadensanzeigen etc.

Die Formular-Metapher ist für Anwendungen geeignet, die



- Textdaten zur Erfassung, Bearbeitung und Ansicht bereitstellen;
- auf effiziente Bearbeitung einer Vielzahl von Daten ausgerichtet sind.

7.7.2 Interaktionsstile

Interaktionsstile strukturieren den Arbeitsablauf im System. Sie geben die Anzahl der Freiheitsgrade vor, mit denen der Benutzer im System navigieren kann und sie beeinflussen die grafische Darstellung der zu bearbeitenden Daten. Einzelne Interaktionsstile korrespondieren mit zugehörigen Arbeitsmetaphern.

Objektorientierter Interaktionsstil

In einer objektorientierten Oberfläche⁴⁶ wählt der Anwender zunächst ein Objekt aus und bearbeitet es anschließend.

Die Auswahl der Bearbeitungsschritte erfolgt auch visuell oft nah am Objekt, z.B. mit Hilfe eines Kontext-Menüs. Beispiele für Anwendungen dieses Oberflächen-Typs sind Vektorgrafik-Programme, Textverarbeitungs-Systeme oder digitale Audio-Anwendungen.



BILD 7.29 Objektorientierte GUI

Objektorientierte Oberflächen sind besonders geeignet für Anwendungen mit grafisch repräsentierbaren Daten. Die Arbeitsweise der Programme orientiert sich an der Werkzeug-Material-Metapher.

⁴⁶ Das Adjektiv „objektorientiert“ bezieht sich ausschließlich auf die Arbeitsweise der zugehörigen Programme. Es hat nichts mit der ausgewählten Programmiersprache zu tun, die für die Erstellung der Software verwendet wird. Zwar ist es in der Regel einfacher, objektorientierte Benutzeroberflächen mit einer objektorientierten Programmiersprache zu erstellen. Es ist jedoch ebenso mit strukturierten Programmiersprachen wie z.B. „C“ möglich.

Frage-Antwort-Interaktionsstil

Das System folgt einem fest vorgegebenen, sequenziellen Arbeitsablauf. Der Benutzer wird durch einfache Eingabe-Masken geführt, die einen Anwendungsfall abbilden.

Diese Arbeitsweise entspricht dem Verhalten von

- Host-Anwendungen: Maskeneingabe – Datenfreigabe – Maskenantwort ...
- HTML-Internet-Anwendungen: Aufruf einer Seite – Aktion – Antwort.
- Assistenten (Wizards) innerhalb einer Anwendung: Navigation nur mit „Vor“, „Zurück“ und „Abbrechen“ möglich.



BILD 7.30
Frage-Antwort-GUI

Beispiele für derartige Oberflächen sind Internet-Shops oder Assistenten zum Erstellen von Software-Komponenten, die in vielen Software-Entwicklungsumgebungen vorhanden sind. Solche Anwendungsfälle verwenden die Fabrik-Arbeitsmetapher.

Der Einsatz dieses Interaktionsstils ist immer dann vorteilhaft, wenn der Benutzer stark angeleitet werden soll und die Aufgaben formalisiert sind. Im Gegenzug führt eine derartige Benutzerschnittstelle zu einer geringen Flexibilität innerhalb der Anwendung. Das System passt sich der Arbeitsweise der Anwender *nicht* an.

Manchmal zwingen auch die eingeschränkten Ressourcen der Laufzeitumgebung zur Verwendung einer derartigen Oberfläche, obwohl sie für das Einsatzgebiet nicht passend ist. Dies kann z.B. bei Micro-Systemen wie Mobiltelefonen der Fall sein, welche nur über ein sehr kleines Display verfügen und keine externen Zeigergeräte wie eine Maus besitzen.

In einem solchen Fall sollten Sie versuchen, die Flexibilität Ihrer Benutzerschnittstelle durch Abkürzungen und Verzweigungsmöglichkeiten zu erhöhen, um zusätzliche Freiheitsgrade zu erreichen.

Formularbasierter Interaktionsstil

Formularbasierte Oberflächen⁴⁷ stellen ihre Anwendungsdaten ausschließlich als Text dar. Die Daten werden meist als Freitext eingegeben, oder aus einer Menge vorgegebener Alternativen ausgewählt (Combo-Boxen, Listen, Radio-Buttons oder Check-Boxen). Viele derartige System folgen dem Muster

- Suche Daten;
- Stelle Suchergebnis als Tabelle dar;
- Wähle Datensatz aus (Ansicht, Bearbeiten, Löschen).

BILD 7.31 Formularbasierte GUI

Derartige Systeme basieren auf der Formular-Metapher (z.B. Recherche- und Abfrage-Systeme, kommerzielle Informationssysteme⁴⁸ etc.).

Wird die Anwendung von Experten benutzt, so können einzelne Masken sehr viele Felder beinhalten und damit zunächst unübersichtlich wirken. Sie erinnern an Papierformulare oder Host-Masken mit einer Fülle von Eingabefeldern und Shortcuts. Für geübte Anwender ist eine derart komprimierte Eingabe jedoch wesentlich ergonomischer als der (umständlichere) Umgang mit Kontext-Menüs und Zusatzdialogen.

⁴⁷ Siehe auch [Coldewey97] und [Coldewey98].

⁴⁸ Unter kommerziellen Informationssystemen verstehen wir Standard- oder Individuallösungen zur Abbildung von Geschäftsprozessen einzelner Unternehmen, wie z.B. Warenwirtschaftssysteme, Stammdatenverwaltungen, Call-Center-Anwendungen etc.

Masken mit einer großen Informationsfülle sollten mit Hilfe von Shortcuts und einer geeigneten Tabulatorensteuerung mauslos bedienbar sein.



Wir haben Anwendungen mit *objektorientierten* Oberflächen gesehen, in denen Mas-senerfassungen von Belegen durchgeführt werden sollten. Dies ging völlig an den Anforderungen der Anwender vorbei, die sequenziell einen Belegstapel abarbeiteten und dabei ausschließlich die Tastatur benutzten, da sie „blind“ schrieben. Erst eine völlige Umgestaltung dieses Anwendungsteiles auf einen formularbasierten Ansatz brachte letztlich die gewünschte Effizienz, Ergonomie und Benutzer-Akzeptanz.

Menüs und Fenster Interaktionsstil

Die Verwendung von Fenstern, Dialogen und Menüs kennzeichnet die Mehrzahl PC-basierter Anwendungen mit grafischer Benutzeroberfläche⁴⁹. Die Strukturierung der Arbeitsabläufe wird meist in der baumartigen Menü-Struktur abgebildet. Innerhalb des Systems stehen fast immer mehrere Alternativen für den nächsten Arbeitsschritt zur Verfügung. Die Gesamtstruktur der möglichen Arbeitsabläufe gleicht damit einem Netz, in dem es Ein- und Ausstiegspunkte sowie fest definierte Wege gibt.

Die einzelnen Anwendungsfälle des Systems werden jeweils mit einem der übrigen Interaktionsstile umgesetzt. Damit bildet die Menü-Fenster-Oberfläche ein Integrationsmittel für Systeme, deren Anwendungsfälle unterschiedlichen Arbeitsmetaphern folgen.

Kommerzielle Informationssysteme setzen die Menü-Fenster-Technik fast immer in Kombination mit formularbasierten Dialogen ein.

Finden Sie passende Metaphern und Interaktionsstile



- Wählen Sie für jeden Anwendungsfall die passende Arbeitsmetapher.
- Kategorisieren Sie die Anwendungsfälle im Hinblick auf die Metaphern.
- Wählen Sie zu den Metaphern passende Interaktionsstile aus:
 - Objektorientierte Oberflächen für die Werkzeug-Material-Metapher,
 - Frage-Antwort-Oberflächen für die Fabrik-Metapher,
 - Formularbasierte Oberflächen für die Formular-Metapher.
- Folgen die Anwendungsfälle Ihres Systems verschiedenen Metaphern, so stimmen Sie Ihr System bewusst darauf ab. Versuchen Sie nicht, alle Anwendungsfälle künstlich auf die gleiche Metapher „zu trimmen“. Dies führt zu umständlicher und wenig intuitiver Navigation und vielen Anwenderfehlern.
- Führen Sie unterschiedliche Interaktionsstile einzelner Anwendungsfallklassen unter einer Menü-Fenster-Oberfläche zusammen.

⁴⁹ im Englischen auch als WIMP bezeichnet: „Windows-Icons-Menus-Pointers“ bzw. „Windows-Icons-Mice-Pulldown menus“.

7.7.3 Ergonomische Gestaltung

Bei der menschlichen Wahrnehmung und Informationsverarbeitung spielen Form und Struktur eine große Rolle. Visuelle Gestaltungsmittel haben gegenüber verbalen Beschreibungen den Vorteil, dass sie bei gleichem Platzbedarf

- eine weitaus höhere Informationsdichte besitzen;
- der assoziativen Verarbeitung entgegen kommen;
- Struktur und Semantik einfacher darstellen können.

Neues wird assoziativ verarbeitet und in das eigene Wissensnetz eingefügt. Überschaubare Einheiten können auf einen Blick erfasst werden. Bei der Gestaltung von Benutzerschnittstellen spielt deshalb die *7 plus/minus 2-Regel* eine wichtige Rolle:

Stellen Sie 5 bis 9 Details innerhalb einer Gruppe dar.

Auswahl der Gestaltungsmittel

Bei der Auswahl eines Darstellungsmittels sollten Sie dessen implizite Bedeutungen bewusst einsetzen. Betrachten Sie die verschiedenen Darstellungen eines Teams in Bild 7.32: Auf der linken Seite sind alle Mitarbeiter sternförmig um den Kollegen *Wagner* gruppiert. Dies wird mit gleichartigen Beziehungen assoziiert. Durch die Sonderrolle im Zentrum wird die Bedeutung von *Wagner* als Gruppenleiter hervorgehoben. Auf der rechten Seite wird die Gruppe hingegen in Form einer Hierarchie dargestellt. Auch hier tritt *Wagner* eindeutig als Kopf der Gruppe hervor. Darüber hinaus wird jedoch auch die Bedeutung von *Hartwig* und *Daum* als Teamleiter symbolisiert. Je höher eine Person in der Hierarchie angeordnet ist, desto größer ist ihre Bedeutung. Bei annähernd gleichem Platzbedarf transportiert die rechte Grafik zusätzliche Informationen.

Beim Entwurf der grafischen Benutzerschnittstelle Ihres Systems sollten Sie folgende Heuristiken berücksichtigen:



- Stellen Sie die wichtigste Information im oberen Teil ihrer Masken dar.
- Gruppieren Sie inhaltlich zusammengehörige Dinge oder Daten mit ähnlicher Änderungsrate nah beieinander.
- Richten Sie die einzelnen Widgets links- oder rechtsbündig an einem unsichtbaren Raster aus.
- Visualisieren Sie Strukturen durch Bäume, Netze etc.
- Ersetzen Sie verbale Beschreibungen durch Symbole, falls dies möglich ist (z.B. Stop-Zeichen für Fehler). Beachten Sie jedoch, dass Symbole oft nicht kulturübergreifend verständlich sind.
- Verwenden Sie sinnvolle Eingabe-Vorbelegungen (Defaultwerte). Unterlassen Sie dies jedoch, wenn es keinen „Favoriten“ gibt. Ein unpassender Defaultwert ist für den Anwender mit Mehrarbeit gegenüber einem leeren Eingabefeld verbunden.*

* Erkennen des unpassenden Wertes, Löschen desselben und anschließende Eingabe der korrekten Daten.

- Lassen Sie sich bei der Gestaltung der Oberfläche von anderen Programmen, die für ähnliche Aufgabenstellungen gedacht sind, inspirieren.**
- Folgen Sie den bewussten und unbewussten Erwartungen der Anwender. Dies macht die Benutzerschnittstelle durch Wiedererkennung intuitiv und leicht erlernbar.
- Überprüfen Sie das GUI-Design so früh wie möglich an Hand von Anwender-Workshops.

**Viele Entwickler kopieren beim Oberflächendesign das, was ihnen bekannt und vertraut ist: Software-Entwicklungstools. Oftmals ist deren GUI-Design jedoch nicht für die Domäne Ihrer Anwendung geeignet.

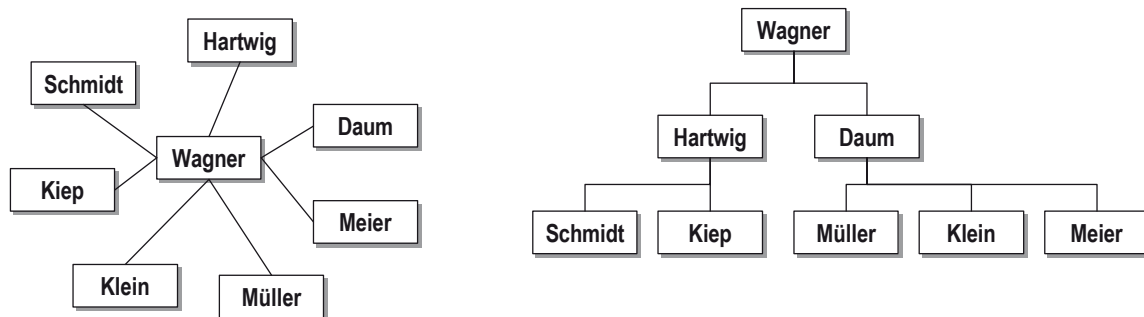


BILD 7.32 Darstellung von Strukturen

7.7.4 Heuristiken zur GUI-Gestaltung

Jennifer Tidwell hat in [Tidwell] eine Patternsprache zur ergonomischen Gestaltung von Benutzerschnittstellen entworfen. Die folgenden Heuristiken basieren auf diesen Entwurfsmustern.

Verdichtete Information

Darstellung einer Vielzahl von *gleichartigen* und *gleich bedeutsamen* Daten. Der Anwender möchte sowohl den Überblick haben als auch Detailinformationen sehen.

Beispiele: Fahrplan, Landkarte, Organigramm

- Stellen Sie möglichst alle Informationen innerhalb *einer* Ansicht dar.
- Sparen Sie Platz durch Verwendung eines kleinen Fonts.
- Setzen Sie Farbe und Schrifttypen (z.B. fett, kursiv) zur Strukturierung ein. Der Anwender kann sich so schneller in der Informationsfülle orientieren.
- Wählen Sie eine grafische Darstellung, die der Informationsstruktur entspricht.

Tabelle

Darstellung von verdichteten Daten in *tabellarischer* Form. Der Anwender möchte viele Informationen auf einmal sehen und Vergleiche anstellen. Ein einzelner Eintrag muss leicht aufzufinden sein.

Beispiele: Adressbuch, Stückliste, Suchergebnis

- Sortieren Sie die Daten nach logischen Kriterien.
- Erleichtern Sie das Auffinden einzelner Werte durch optische Anker (z.B. bei einer alphabetischen Sortierung durch Fettdruck für den ersten Eintrag eines neuen Buchstabens).
- Ordnen Sie die Spalten in absteigender Wichtigkeit von links nach rechts.

Hierarchie

Darstellung verdichteter, *hierarchisch geordneter* Daten. Das Wissen über die Informationsstruktur ist für den Anwender von großer Bedeutung. Es hilft ihm, Daten schnell wiederzufinden und Beziehungen zu erkennen.

Beispiele: Verzeichnisse (Dateien, Warengruppen etc.), Organigramm

- Stellen Sie die Daten in einer Baumstruktur dar.

Formular

Standardisierte Dateneingabe.

Beispiele: Internet-Bestellung, Buchungssatz anlegen, Schadensanzeige

- Heben Sie Pflichteingaben von optionalen Feldern optisch hervor (z.B. durch die Hintergrundfarbe).
- Ausgabefelder müssen von Eingabefeldern klar zu unterscheiden sein.
- Machen Sie deutlich, welche Eingaben erwartet werden. Dies kann durch vorformatierte Felder oder Beispiele erfolgen.

Expertensystem

Anwendung mit *langer Lebenszeit*, die von den Anwendern zur Bearbeitung *komplexer Aufgaben* verwendet wird. Der Benutzer will eine Vielzahl von Daten gleichzeitig sehen und bearbeiten. Längere Einarbeitungszeiten werden akzeptiert.⁵⁰

Beispiele: Tabellenkalkulation, Bildbearbeitung, Audio-Schnittplatz

- Lassen Sie viel Platz für die Darstellung der Arbeitsgegenstände.
- Platzieren Sie Statusinformationen und Werkzeuge an den Rändern.
- Die Aufgaben müssen *performant* und mit möglichst wenigen Arbeitsschritten zu erledigen sein.
- Bieten Sie Möglichkeiten zur benutzerspezifischen Konfiguration.

⁵⁰ Die Einarbeitungszeit ist minimal im Vergleich zur späteren produktiven Einsatzzeit der Anwendung.

Strukturelle Wiederholung

Darstellung und Navigation folgen stets dem gleichen visuellen und strukturellen Muster. Durch die permanente Wiederholung werden die Bedeutungen implizit erfasst und die Bedienung intuitiv.

Beispiele: Buch mit gleichartigem Layout aller Kapitel, Standard-Menüs und Standard-Dialoge in Windows-Programmen

- Verwenden Sie Symbole und Dialoge aus dem Betriebssystem-Standard, wann immer dies möglich ist.
- Bilden Sie gleichartige Aufgaben im gleichen Interaktionsstil ab.
- Führen Sie Standards für Buttons und Kontext-Menüs ein (Kombination, Anordnung, Beschriftungen, Default-Button).
- Verwenden Sie gleiche Fonts und Farben zur Hervorhebung gleicher Bedeutung (Beschriftung, Erläuterung, Kontext der Bearbeitung etc.).

Gruppe verwandter Dinge

Darstellung zusammenhängender Informationen (nach der 7+/-2-Regel). Kleine Informationseinheiten können vom Anwender schnell erkannt werden.

Beispiele: Adressblock auf einer Kunden-Maske

- Verwenden Sie visuelle Muster für die Anordnung der Gruppen.
- Setzen Sie die Gruppen durch breitere Zeilen- oder Spaltenabstände voneinander ab.
- Beschränken Sie den Einsatz von Gruppierungsrahmen.

Rückkehr an einen sicheren Ort

Rückkehr zu einem konsistenten *Ausgangspunkt*. Der Anwender kann sich in der Anwendung „verirrt“ haben.

Beispiele: *Home*-Button im Web-Browser, *Zurücksetzen*-Button in Eingabemasken, *Standard*-Button in Konfigurationsmasken

- Bieten Sie eine Rückkehrmöglichkeit zum letzten konsistenten Zustand an (gesicherte Datei, aktuelle Konfiguration etc.).

Statusanzeige

Darstellung eines *Zustands*. Die Information ist für den Anwender nur gelegentlich interessant. Sie muss einfach zu finden sein, darf aber die Haupttätigkeit nicht stören.

Beispiele: Uhrzeit, Tachometer, Akku-Ladezustand, System-Ressourcen

- Verwenden Sie Symbole.
- Betonen Sie Wichtiges (z.B. durch Farbe).
- Stellen Sie die Information immer an der gleichen Stelle dar. Behalten Sie bei mehreren Statusinformationen stets die gleiche Anordnung bei.

Fortschrittsanzeige

Darstellung des *Fortschritts der aktuellen Operation*. Der Anwender möchte wissen, ob das System noch arbeitet. Sobald auf dem Bildschirm kein Fortschritt angezeigt wird, ist der Benutzer beunruhigt, dass die Anwendung hängen könnte.

Beispiele: Datei-Download, Installations-Prozedur

- Sorgen Sie dafür, dass sich auf dem Bildschirm etwas verändert (Textausgabe, Fortschrittsbalken etc.).
- Geben Sie an, wie groß der Anteil der bereits erledigten Aufgaben ist und wie viel voraussichtlich noch zu tun ist.

Mehrfach-Aktion

Auf mehrere gleichartige Objekte soll die gleiche Operation ausgeführt werden. Der Anwender möchte die einzelnen Bearbeitungsschritte nicht für jedes einzelne Element wiederholen⁵¹.

Beispiele: Löschen mehrerer Dateien, Verschieben mehrerer Objekte

- Wenn eine Operation dies zulässt, so geben Sie die Möglichkeit zur Mehrfachauswahl von Objekten.

7.7.5 Weiterführende Literatur



www.uidesign.net bietet viele Ressourcen zum Thema Benutzerschnittstelle.

[HalShm] ist eine umfangreiche Sammlung gelungener und abschreckender Beispiele zu den Themen GUI-Design und Benutzer-Interaktion.

[Shneidermann98] enthält viele praktische Hinweise zum Entwurf von Benutzerschnittstellen.

[Tidwell] beschreibt eine Entwurfsmuster-Sprache für Benutzerschnittstellen.

⁵¹ Maschinen eignen sich weitaus besser für die stupide Wiederholung gleicher Tätigkeiten. Menschen ermüden dabei und arbeiten schließlich fehlerhaft.

■ 7.8 Internationalisierung

von Kerstin Dittert

Internationalisierung: Unterstützung für den Einsatz von Systemen in unterschiedlichen Ländern, Anpassung der Systeme an länderspezifische Merkmale.

Lange bevor Softwareprodukte international eingesetzt werden sollten, entwickelte die Industrie schon Produkte für globale Märkte. Stellen Sie sich folgendes Szenario vor:

Ein Automobilhersteller des europäischen Fantasielandes Expando vertreibt einen PKW, der sich im Heimatland zum Verkaufsschlager entwickelt. Das Management beschließt, den Wagen auf den internationalen Markt zu bringen. Ein Team erhält den Auftrag, festzustellen, welche Ergänzungen notwendig sind, um das Fahrzeug international zu vertreiben. Die Gruppe erstellt in langwieriger Detailarbeit einer Liste aller Beschriftungen im Wageninneren. Sie fordern eine Übersetzung dieser Texte und der Betriebsanleitung in alle europäischen Sprachen an. Mit beträchtlichem Aufwand werden die Texte übersetzt. Der Wagen lässt sich nun für jedes Zielland neu konfigurieren:

- Alle Beschriftungen sind in der Landessprache verfasst.
- Das eingebaute Navigationssystem ist für jedes Land mit den entsprechenden Karten und der passenden Sprachausgabe ausgestattet.
- Für jedes Land gibt es eine andere Betriebsanleitung.

Das Marketing beginnt, der Wagen wird europaweit und in Amerika vertrieben. Zur Überraschung aller entwickelt sich der ausländische Umsatz mehr als schleppend. In England wird kein einziges Auto verkauft. In Deutschland veröffentlicht der ADAC Testergebnisse, nach denen der Wagen bei 180 km/h in der Kurve ausbricht. In Amerika wird der Wagen gar nicht erst für den Verkauf zugelassen! Das Management beruft eine Krisensitzung ein. Es war doch so viel Aufwand in die Übersetzungsarbeit gesteckt worden! Wieso nahmen die Kunden den Wagen nicht an? Eine in Auftrag gegebene Studie bringt ernüchternde Ergebnisse zu Tage:

- In England hatte man vergessen, das Lenkrad *rechts* einzubauen.
- In der amerikanischen Version hatte man vergessen, den Tacho auf die *Maßeinheit* Meilen/Stunde umzurechnen.
- Da in Expando das Tempolimit 130 km/h gilt, wurde die Konstruktion auf diese Höchstgeschwindigkeit ausgelegt. Der Fehler wurde von der Qualitätssicherung nicht entdeckt, da *spezielle Tests* für einzelne Länder-Konfigurationen nicht durchgeführt wurden!
- Bei der Konzentration auf die sprachlichen Aspekte hatte man andere Aspekte wie Maßeinheiten, technische Randbedingungen etc. völlig vergessen.

7.8.1 Globale Märkte erfordern neue Prozesse

So überspitzt die obige Geschichte auch sein mag: Die Erstellung international einsetzbarer Software stellt Sie als Software-Architekten vor ähnliche Probleme. Der Aspekt der Internationalisierung beeinflusst den gesamten Softwareentwicklungsprozess. Sie sollten ihn deshalb frühzeitig in Ihre Überlegungen mit einbeziehen. Neben der Berücksichtigung verschiedener Sprachen müssen Sie manchmal auch nationale Unterschiede wie Währungen, gängige Formatierungen, Zeitzonen etc. berücksichtigen.

Je nach Zielgruppe und angestrebtem Markt kann es sogar erforderlich sein, kulturelle und politische Aspekte mit zu betrachten. Dies ist besonders wichtig für Web-Anwendungen, die sich an ein weltweites und unspezifisches Publikum wenden (wie z.B. Suchmaschinen, Free-Mail-Provider etc.).

7.8.2 Dimensionen der Internationalisierung

Internationalisierung ist teuer, und nicht jede Anwendung ist für den chinesischen Markt mit einer Fülle von über 40000 Schriftzeichen geplant. Allgemeine generische Ansätze sind aufwändig in der Entwicklung, ohne dass damit zwangsläufig ein wirtschaftlicher Mehrwert verbunden ist. Zu Beginn des Architekturentwurfs sollten Sie deshalb die Anwender-Zielgruppe möglichst genau eingrenzen. Dabei sollten Sie folgende Fragen stellen:

- Welche Sprachen sollen unterstützt werden?
- Verwenden alle Sprachen ausschließlich lateinische Schriftzeichen?
- Wird Text immer horizontal von links nach rechts geschrieben?
- In welchen Ländern soll die Anwendung eingesetzt werden?

Sie und Ihr Team müssen herausfinden, wie hoch der Anteil sprachabhängiger oder landestypischer Anwendungslogik ist. Folgende Fragen können Ihnen dabei helfen:

- Welche Formatierungen werden benötigt?
- Müssen unterschiedliche Währungen verarbeitet werden?
- Enthalten die Anwendungsdaten Bezeichnungen, die übersetzt werden müssen?
- Enthält die Anwendung zeitabhängige Dienste?
- Welchen Umfang soll das Hilfesystem haben?
- Welche Anwendungsteile sollen über Tastaturkürzel⁵² gesteuert werden?
- Gibt es kulturelle Unterschiede, die für die Anwendung relevant sind?

Haben Sie diese Fragen beantwortet, können Sie entscheiden, welche der folgenden Konzepte für Ihre Architektur relevant sind. Als wichtigste Regel gilt:



Grenzen Sie die Internationalisierungsaspekte so weit wie möglich ein.

⁵² Shortcuts (z.B. CTRL-C) oder Mnemonics (ALT-Buchstabe).

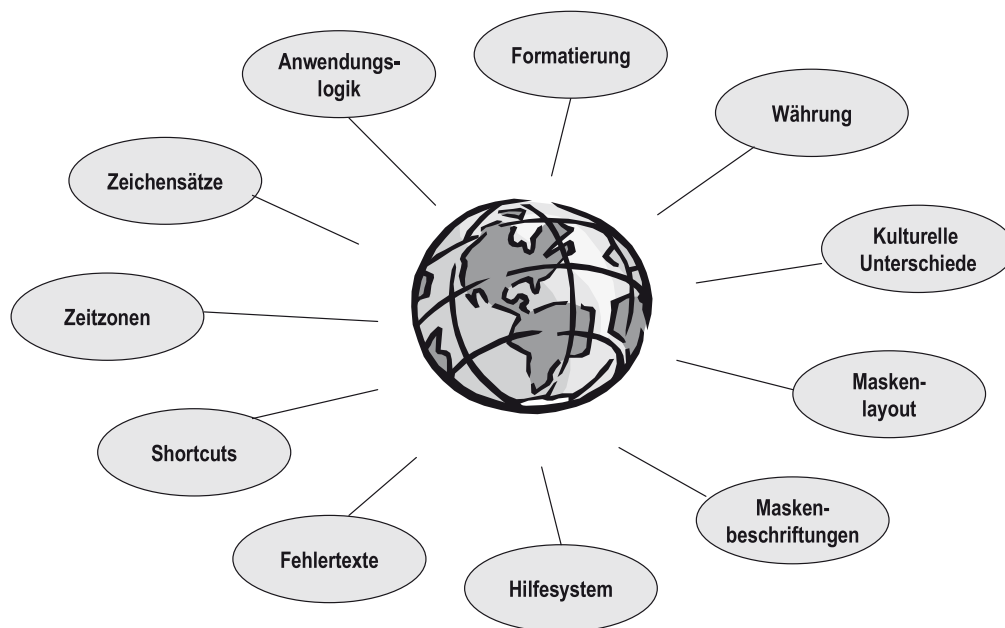


BILD 7.33 Aspekte der Internationalisierung

7.8.3 Lösungskonzepte

Mehrsprachigkeit

Sprachen
Sprach-
varianten

Fast alle internationalisierten Anwendungen unterstützen mehrere Sprachen, manche sogar Sprachvarianten wie z.B. *Britisches* und *Amerikanisches Englisch*. Die Notwendigkeit solch feiner Unterscheidung müssen Sie sorgfältig abwägen. Die Ziele einer hohen Anwenderakzeptanz und des möglichst geringen Entwicklungs- und Wartungsaufwandes stehen sich komplementär gegenüber. Für die Rechtschreibprüfung einer Textverarbeitung ist die Berücksichtigung von Sprachvarianten unerlässlich. Die Webseiten einer Internet-Suchmaschine können hingegen ausschließlich in einer englischen Sprachvariante angeboten werden.

Externe
Ressourcen

Sämtliche Oberflächenelemente müssen sprachabhängig beschriftet werden und werden deshalb als externe Ressourcen benötigt. Rückmeldungen an den Anwender wie Fehlermeldungen, Statuszeilentexte etc. müssen ebenfalls in allen Sprachen vorliegen und beim Satzbau den Regeln der jeweiligen Grammatik folgen.

Textkonstanten sind in verschiedenen Sprachen unterschiedlich lang und beanspruchen dementsprechend mehr oder weniger Platz. Grafische Benutzeroberflächen mehrsprachiger Systeme sollten deshalb niemals ein statisches GUI-Layout⁵³ vorgeben. Stattdessen sollte sich die Größe beschrifteter Elemente (Buttons, Label, Menüeinträge etc.) dynamisch an den enthaltenen Text anpassen.

⁵³ Bei einem statischen GUI-Layout sind die Position und Größe der einzelnen Oberflächenelemente fest vorgegeben. Bei unbekannten Beschriftungslängen muss also vorsorglich einiges an zusätzlichem Platz reserviert werden, damit die Texte nicht abgeschnitten werden.

Die Oberflächenorientierung einer GUI folgt immer der Textausrichtung der Sprache. Menüs werden z.B. in deutschsprachigen Ländern von links nach rechts angeordnet, in arabisch sprechenden Ländern jedoch von rechts nach links. Auch werden die Elemente einer Maske entweder von links nach rechts oder umgekehrt angeordnet. Falls also Sprachen mit unterschiedlicher Textausrichtung unterstützt werden sollen (z.B. Englisch und Arabisch), so wird eine variable GUI-Komponentenorientierung benötigt. Hierfür muss das GUI-Layout-Management neben der variablen Positionierung und Größenbestimmung um dynamische Orientierung ergänzt werden.

Die folgende Heuristik kann Ihnen helfen, Ihre Architektur für mehrere Sprachen auszulegen:



- Legen Sie zunächst die für das System erforderlichen Sprachen fest (z.B. Deutsch, Englisch, Arabisch etc.).
- Überprüfen Sie sorgfältig, ob Sie unterschiedliche Sprachvarianten benötigen (z.B. Deutsch (Schweiz), Deutsch (Deutschland)). Der Übersetzungsaufwand steht häufig in keinem vernünftigen Verhältnis zum Mehrwert innerhalb der Anwendung!
- Setzen Sie bei der Masken-Gestaltung Layout-Manager-Klassen ein. Diese passen die Größe der Maskenelemente an die aktuellen Beschriftungslängen an.
- Sollte die von Ihnen eingesetzte Programmiersprache kein dynamisches Layout-Management unterstützen, so berücksichtigen Sie zusätzlichen Platzbedarf für andere Sprachen. Geben Sie in Entwurfsrichtlinien maximale Beschriftungslängen vor (z.B. 30 Zeichen).
- Ordnen Sie Menüs und andere Oberflächenelemente gemäß der Textausrichtung der verwendeten Sprache an.
- Legen Sie sämtliche sprachabhängigen Elemente in Ressource-Dateien ab (Oberflächenbeschriftungen, Fehlertexte, Shortcuts, Tooltips, Audio-Dateien etc.).
- Beachten Sie bei Fehlermeldungen den unterschiedlichen Satzbau verschiedener Sprachen. Einige Klassenbibliotheken bieten hierfür spezielle Formatierungsklassen an.*

* In Java ist dies die Klasse `MessageFormat`

Anbindung externer Ressourcen

Sämtliche statischen Texte und alle übrigen sprachabhängigen Inhalte, wie Bilder oder Audio-Dateien, werden in sprachspezifischen Ressource-Dateien oder -Klassen von der Anwendungsimplementierung entkoppelt. Zur Laufzeit bestimmt das Programm die aktuelle Kombination von Land und Sprache. Dies kann mit Betriebssystemmitteln, über ein Benutzerprofil oder auf Anforderung des Benutzers erfolgen. Diese Länder-Sprachkombination wird allen internationalisierten Anwendungsteilen zur Verfügung gestellt⁵⁴. Anschließend können die jeweils benötigten Ressourcen geladen werden.

⁵⁴ Sie wird häufig als `Locale`-Objekt bezeichnet.